

RAPORT PROIECT: 4 WAY SA CACHE CONTROLLER

Vlonga Dorina Nicoleta, Fizădean Alexandru

C.3.2

1. SPECIFICAȚII DE PROIECTARE

1.1 Alegerea limbajului hardware

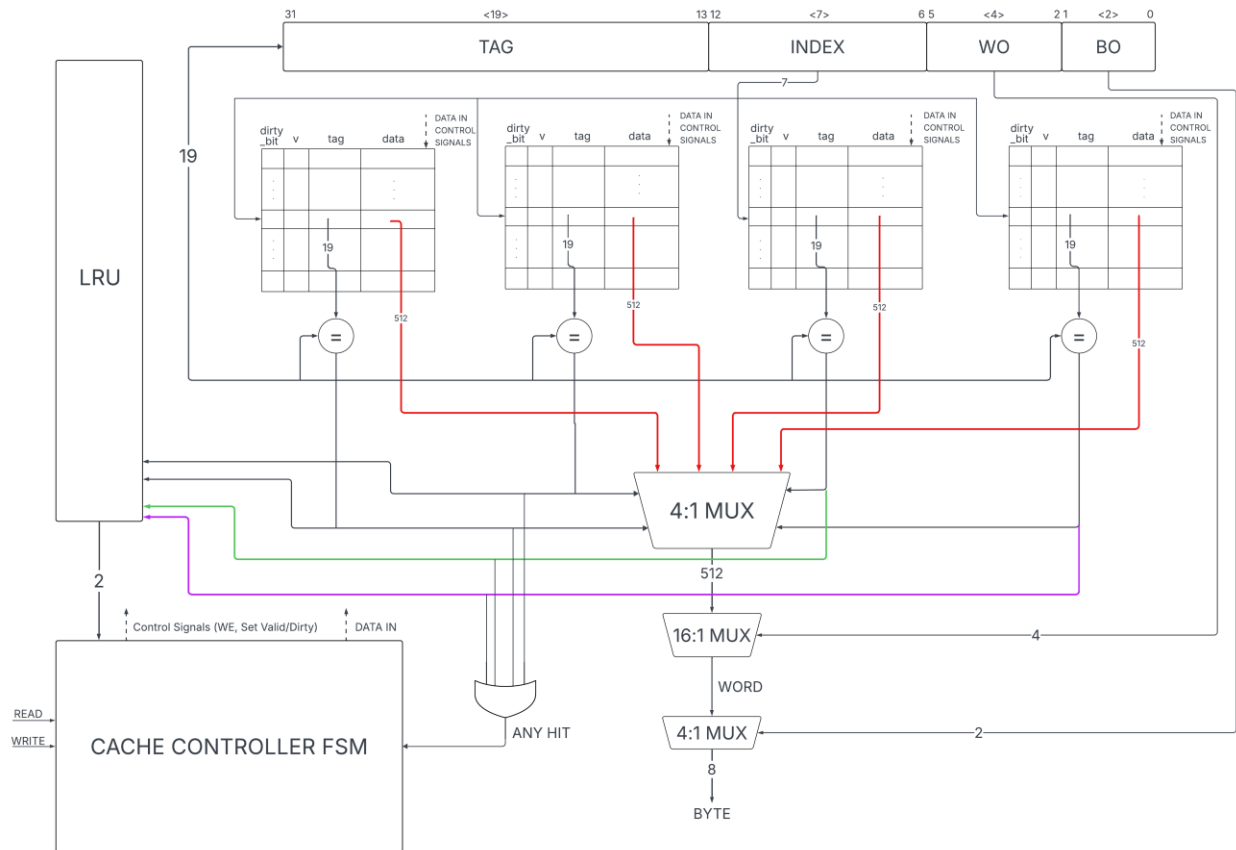
Pentru acest proiect am ales să scriem codul în **Verilog**. Motivul principal este că am mai lucrat cu el (la implementarea aceluia ALU pe 8 biți) și ne-am obișnuit cu sintaxa. E mult mai simplu să declari blocurile de memorie ca niște vectori în Verilog, iar structura cu blocuri de tip `always` ne-a ajutat să separăm clar logica combinațională de cea secvențială pentru automatul de stări (FSM).

1.2 Organizarea adreselor și arhitectura

Avem de făcut o memorie cache de 32 KB, organizată pe 4 căi (4-way set asociative). Dimensiunea unui bloc este de 64 de bytes, iar cache-ul are 128 de seturi. Când procesorul ne trimite o adresă pe 32 de biți, noi o împărțim în felul următor:

- **Byte Offset (2 biți):** Ne ajută să alegem exact un byte din cei 4 ai unui cuvânt (biții [1:0]).
- **Word Offset (4 biți):** Un bloc de 64 bytes are 16 cuvinte. Cu 4 biți alegem unul din aceste 16 cuvinte (biții [5:2]). În total, offset-ul are 6 biți.
- **Index (7 biți):** Cu acești biți selectăm unul dintre cele 128 de seturi (biții [12:6]).
- **Tag (19 biți):** Restul de biți rămași din adresă ($32 - 7 - 6 = 19$, biții [31:13]).

BLOCK DIAGRAM



Pe diagrama bloc se poate vedea că am folosit 4 matrice paralele pentru cele 4 căi. Fiecare are un bit de Valid, un bit de Dirty (pentru Write-Back), Tag-ul și Datele. Când primim o adresă, comparăm Tag-ul pe toate cele 4 căi în același timp. Dacă avem o potrivire, un prim multiplexor (MUX 4:1) alege blocul întreg de 512 biți, iar apoi un MUX 16:1 extrage doar cuvântul de 32 de biți pe care îl vrea procesorul.

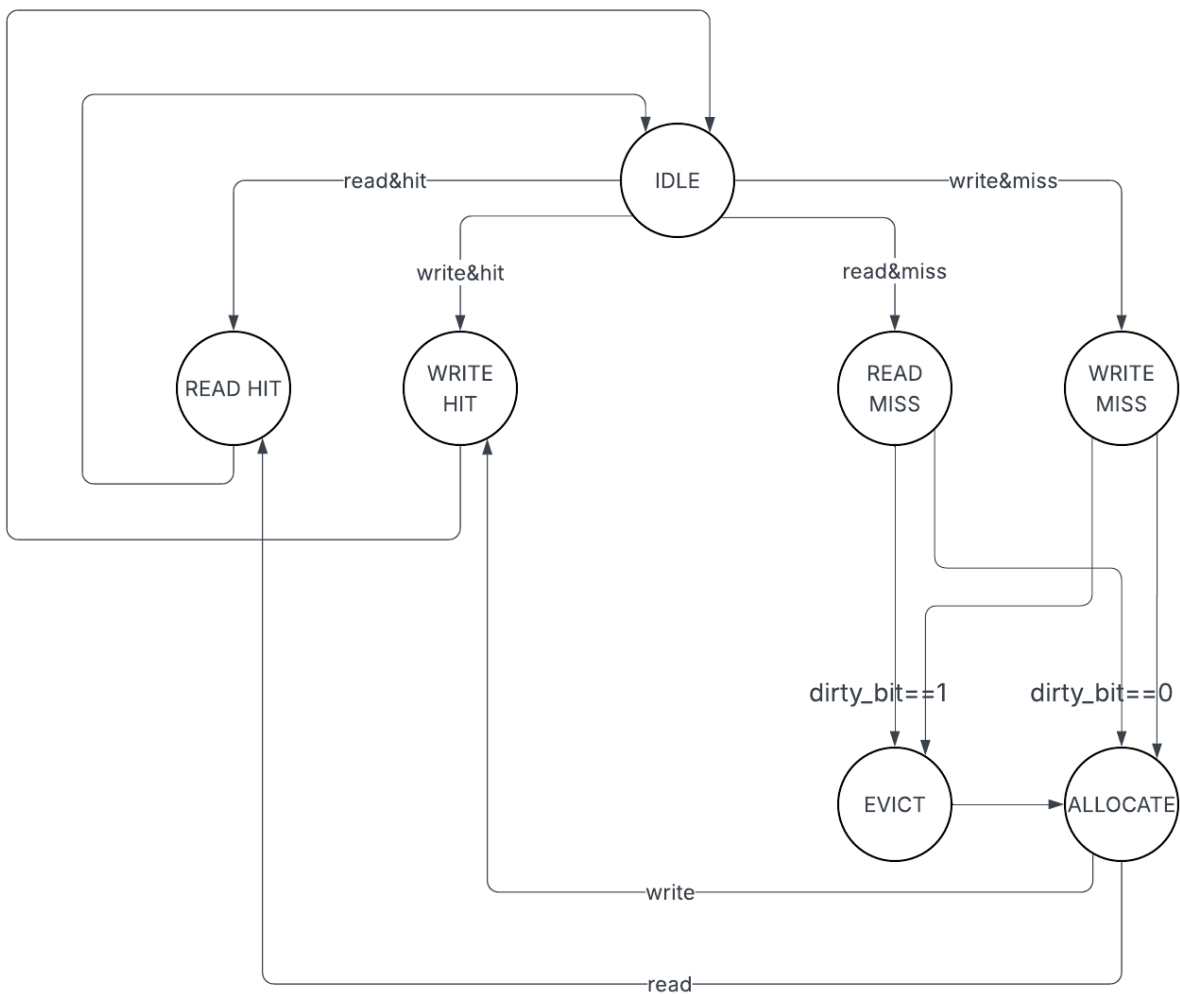
1.3 FSM-ul (Automatul de control)

Pentru a controla tot sistemul, am desenat un FSM cu 7 stări:

1. **IDLE:** Starea de bază, stăm și așteptăm comenzi de Read sau Write de la procesor.
2. **READ_HIT:** Am găsit datele în cache, le trimitem imediat la procesor.
3. **WRITE_HIT:** Scriem noul cuvânt în cache și facem bitul dirty = 1.

4. **READ_MISS / WRITE_MISS:** Ne dăm seama că datele nu sunt în cache. Ne uităm la contoarele LRU ca să vedem pe cine dăm afară. Dacă blocul victimă are dirty=1, mergem în EVICT. Dacă are dirty=0, mergem direct în ALLOCATE.
5. **EVICT:** Trimitem blocul modificat înapoi în memoria RAM ca să nu pierdem datele.
6. **ALLOCATE:** Aducem blocul nou din RAM în cache, apoi ne întoarcem să rezolvăm comanda inițială a procesorului.

FSM diagram



2. IMPLEMENTARE HARDWARE

Codul sursă a fost împărțit logic în mai multe secțiuni. Am inițializat memoriile, am făcut logica pentru Hit/Miss privind reperele din schemă și am scris un bloc separat pentru actualizarea vârstei blocurilor (lru_cnt).

3. PLAN DE TESTARE ȘI REZULTATE

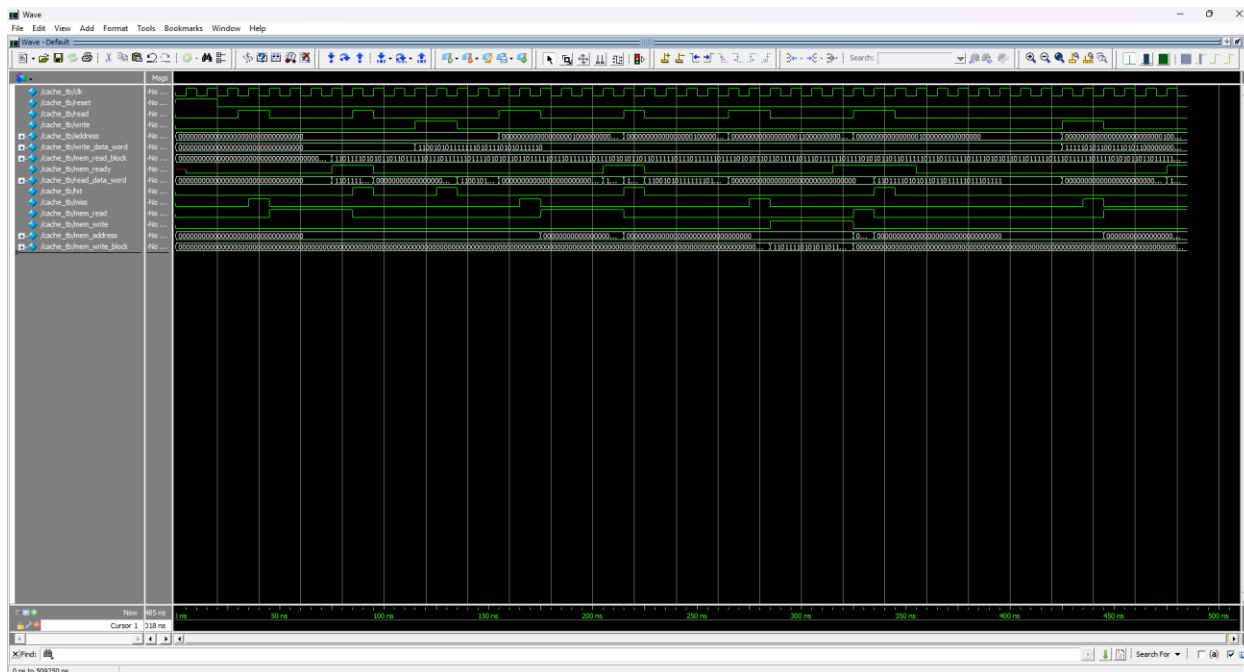
3.1 Scenariile de test

Ca să fim siguri că FSM-ul trece prin toate stările, am scris un testbench care simulează și cum răspunde memoria RAM și am rulat 5 teste:

- **Test 1 (Read Miss):** Am cerut adresa 0. Cache-ul e gol, deci dă miss, trece prin ALLOCATE și aduce datele.
- **Test 2 (Read Hit):** Am cerut din nou adresa 0. De data asta a dat hit instantaneu.
- **Test 3 (Write Hit):** Am scris ceva la adresa 0. Cache-ul a făcut bitul dirty 1.
- **Test 4 (Umplerea cache-ului):** Am citit încă 3 adrese diferite care pică pe același Index, ca să umplem toate cele 4 căi disponibile.
- **Test 5 (Evict):** Cel mai important test. Am cerut o a 5-a adresă pe acel Index. Cache-ul nu mai avea loc, așa că LRU a ales să dea afară Calea 0 (cea mai veche). Pentru că era "murdară" de la Testul 3, FSM-ul s-a dus în EVICT, a salvat datele în RAM, și abia apoi a adus noul bloc.
- **Test 6 (Write Miss):** Am cerut o operație de scriere la o adresă complet nouă (0x00000040). FSM-ul trece prin starea WRITE_MISS, apoi în ALLOCATE (pentru a aduce blocul nou) și se întoarce automat în WRITE_HIT pentru a scrie cuvântul dorit.

3.2 Forme de undă în ModelSim

Am deschis proiectul în ModelSim și am urmărit semnalele. Din captura de ecran se vede clar cum la Testul 5 (pe la momentul 330ns), semnalul mem_write trece pe 1 ca să salveze blocul evacuat, așteaptă mem_ready, iar imediat după, se activează mem_read pentru a aduce noul bloc. Nu am avut stări blocate sau fire lăsate în aer.



3.3 Rata de hit și timpii de acces

- **Hit Rate:** În testul nostru am avut **8 accesări** în total (2 hit-uri și 6 miss-uri). Asta dă un Hit Rate de **25%**. Evident, procentul e mic doar pentru că noi am forțat cache-ul să dea rateuri ca să vedem dacă funcționează stările de Evict și Allocate. Într-un program real, ar trece de 90%.
- **Timp de acces:** Când dă Hit, durează doar 1 ciclu de ceas. Când dă Miss (curat), durează cam 4 cicluri (depinde de cât îi ia RAM-ului să răspundă). Dacă trebuie să facem și Evict, durează cel mai mult, cam 7-8 cicluri, pentru că scriem în RAM și apoi citim.

4. RAPORT FINAL: CONCLUZII ȘI PROVOCĂRI

Pentru realizarea acestui proiect, am pornit inițial de la partea teoretică și de la schițele pe foaie. Am calculat exact câți biți ne trebuie pentru offset-uri și index, după care am trecut la desenarea diagramei bloc. A ajutat foarte mult faptul că am vizualizat clar cum datele ies din cele 4 matrice și trec prin multiplexoare înainte să ne apucăm de scris efectiv cod Verilog. După ce am clarificat comment-urile legate de FSM și am stabilit condițiile clare pentru tranziții (în special la verificarea bitului dirty), implementarea a mers mult mai logic.

Totuși, în faza de scriere a codului și la simulare, ne-am lovit de câteva provocări tehnice specifice. Prima și cea mai evidentă a fost lucrul cu memoria RAM. Memoria principală este

mult mai lentă decât cache-ul, deci nu puteam pur și simplu să trecem dintr-o stare în alta într-un singur ciclu de ceas. A trebuit să simulăm un mecanism de handshake, folosind semnalul `mem_ready`. FSM-ul nostru este blocat în stările de `ALLOCATE` sau `EVICT` până când RAM-ul semnalizează că a terminat de citit sau de scris datele. Fără acest semnal, cache-ul nostru s-ar fi desincronizat total.

A doua mare bătaie de cap a fost implementarea politicii "Write Allocate". La început, ne-am gândit că dacă procesorul vrea să scrie ceva și nu găsește adresa în cache (Write Miss), ar putea să scrie direct în RAM. Dar cerința cerea Write Allocate, ceea ce înseamnă că a trebuit să forțăm FSM-ul să aducă mai întâi tot blocul de 64 de bytes din memoria principală, să-l pună în cache, și abia apoi să suprascrie cuvântul dorit. A fost un pic contraintuitiv la început, dar am înțeles că așa se păstrează coerența datelor.

Nu în ultimul rând, actualizarea contoarelor pentru algoritmul LRU ne-a dat ceva erori la primele rulări. Când aveam un Hit, dacă pur și simplu scădeam valoarea tuturor celorlalte contoare, ajungeam în situația în care mai multe căi aveau vârsta "0", iar cache-ul nu mai știa pe cine să evacueze. Soluția a fost să punem niște condiții clare: doar căile care erau mai noi decât calea curent accesată se decrementează, în timp ce calea folosită primește mereu prioritatea maximă (valoarea 3). Așa am evitat coliziunile.

În concluzie, simulările din ModelSim ne-au dovedit că sistemul este stabil și face exact ce trebuie. Am înțeles practic de ce politica Write-Back este atât de utilă: salvează enorm de mult timp pe magistrala sistemului, pentru că scrie în RAM doar datele care chiar au fost modificate, și abia în ultima clipă, când blocul respectiv e dat afară din cache. A fost un proiect destul de complex, dar care ne-a clarificat perfect cum comunică procesorul cu ierarhia de memorie.